

DaF ERP — Moliyaviy Tizim

To'liq Texnik Tahlil

Hujjat versiyasi: 4.0 (post Faza 8 audit)

Sana: 2026-04-30

Kapital: Bu hujjat backend va frontend kodlarini, hozirgi commit qilinmagan o'zgarishlarni va yangi migrationlarni o'rganib chiqilgan tahlildir.

Asos manbalar: server/src/billing/*, server/src/salary/*, server/src/refunds/*, server/src/students/*, server/src/transactions/*, server/src/payments/*, server/src/lesson-cancellations/*, server/src/student-exit-reasons/*, server/prisma/schema.prisma va so'nggi 4 migration, client/src/components/payments/*, client/src/components/profile/*, client/src/components/students/*, client/src/components/teachers/*, client/src/components/groups/*.

Tahlil maqsadi: Foydalanuvchi (CEO) tomonidan so'ralgan kengaytirilgan, holat-asosli tushuntirish — har bir mumkin bo'lgan stsenariy uchun "agar A holat bo'lsa, X sodir bo'ladi; agar B holat bo'lsa, Y sodir bo'ladi" ko'rinishida.

Mundarija

1. Asosiy printsiplar va ma'lumotlar bazasi qatlami
2. Pul oqimining umumiy ko'rinishi (real misollar)
3. Davomat va billing oqimi
4. Ustozga oylik hisoblanishi
5. O'quvchi hayot tsikli moliyaviy nuqtai nazardan
6. Pul qaytarish (refund) va to'lov bekor qilish (reverse)
7. Dars bekor qilish (lesson cancellation) va dam olish kunlari
8. Initial Balance va eski tizimdan migratsiya
9. Markaz xarajatlari va TEACHER_ADVANCE
10. Maxsus holatlar (edge cases)
11. Frontend UI yo'llari va sahifalari
12. Rollar va ruxsatlar matritsasi
13. Topilgan kamchiliklar va e'tibor talab qiluvchi joylar

1. Asosiy printsiplar va ma'lumotlar bazasi qatlami

1.1 Append-only Ledger printsipi

Tizimda har bir pul harakati **Transaction** jadvaliga yoziladi. Ushbu yozuvlar **hech qachon o'chirilmaydi yoki tahrir qilinmaydi**. Xato bo'lsa, teskari (reversal) yozuv qo'shiladi va asl yozuvga **reversedAt**, **reversedById**, **reversalReason** belgilari qo'yiladi.

Bu yondashuv quyidagi imkonlarni beradi:

- **Auditga tayyorlik:** har bir tiyinning yo'li tarix orqali kuzatiladi
- **Idempotentlik:** takroriy operatsiyalar partial unique indekslar orqali bloklanadi
- **Reverssibility:** xato qaytarish ham yozuv sifatida saqlanadi (faqat **reversedAt** markerini olib tashlash)

Muhim:

Tizimning ishonchli yagona haqiqat manbai — **Transaction** jadvali. Boshqa jadvallarda saqlanadigan summalar (masalan **Student.balance**, **Contract.paidAmount**) — ledger asosida hisoblangan ko'zgu (mirror).

1.2 Atomiklik va race condition himoyasi

Barcha balans o'zgaruvchi operatsiyalar quyidagi parametrlar bilan o'raladi:

```
prisma.$transaction(callback, {
  isolationLevel: Prisma.TransactionIsolationLevel.Serializable,
  maxWait: 10000,    // 10s – Neon serverless cold-start uchun
  timeout: 15000     // 15s
});
```

Student.balance yoki **User.balance** o'qish va yozish **SELECT FOR UPDATE** bilan qulflanadi — bir vaqtda ishlayotgan ikkinchi tranzaksiya birinchisi yakunlanguncha kutadi.

1.3 Reversal markeri (**reversedAt**)

20260428010035 migration **Transaction** jadvaliga ikkita yangi maydon qo'shdi: **reversedAt** va **reversedById**. **reverseTransaction()** metodi *ikki ish* qiladi:

1. Yangi reversal qator yozadi (asl **reversedTransactionId** bilan bog'langan)
2. Asl qatorga **reversedAt = now()** va **reversedById** yozadi

Barcha "hali aktiv" filtrlar **WHERE reversedAt IS NULL** ishlatadi.

1.4 Multi-tenant izolyatsiya

Har bir moliyaviy yozuvda **companyId** NOT NULL maydoni bor. Barcha read/write operatsiyalar JWT tokenidan olingan **@CurrentUser('companyId')** bo'yicha filtrlash bilan qilinadi. Filial direktori (Branch

Director) qo'shimcha ravishda faqat o'z filiali ma'lumotlarini ko'radi.

1.5 Asosiy moliyaviy modellar

Model	Maqsad	Asosiy maydonlar
Payment	Tashqi pul kelishi (kassa yoki online)	amount, method, status, source, externalId, contractId
Transaction	Bosh ledger — har bir balans o'zgarishi	type, amount, balanceBefore/After, metadata, reversedAt
Contract	O'quvchi-kurs shartnomasi	contractNumber, totalAmount, paidAmount, status
Enrollment	O'quvchi-guruh bog'lanishi	status, prepaidLessonsRemaining, departureReasonId
Attendance	Davomat yozuvi	status (PRESENT/LATE/ABSENT/EXCUSED), markedMethod, cancellationId
EmployeeSalaryConfig	Xodim oylik sozlamasi (ko'zgu — joriy qiymat)	userId, groupId?, salaryType, value, isActive
EmployeeSalaryConfigVersion	Oylik sozlamasi tarixi (SCD2)	configId, salaryType, value, effectiveFrom, effectiveTo
SalaryAccrual	Bitta dars uchun ustozga yig'ilgan summa	userId, studentId, lessonDate, amount, deductionTransactionId, reversedAt
SalaryPayment	Oylik to'lov (CALCULATED → APPROVED → PAID)	userId, periodStart/End, amount, status
SalaryPeriodSetting	Hisoblash davri sozlamasi (default: 8-sana)	cycleStartDay, effectiveFrom/To
Refund	Pul qaytarish so'rovi va jarayoni	requestedAmount, approvedAmount, deductions JSON, status
Expense	Markaz xarajati	category, amount, relatedUserId, settledBySalaryPaymentId
LessonCancellation	Bitta guruhning bitta kunini bekor qilish	groupId, date, reason, deletedAt
StudentExitReason	O'quvchi chiqib ketish sabablari katalogi	name, appliesTo (ExitType[])

1.6 TransactionType enum (10 ta tur)

Tur	Qachon yoziladi	Amount belgisi
PAYMENT	O'quvchi to'lovi (kassa yoki gateway)	+ (balansga qo'shadi)
LESSON_DEDUCTION	Prepaid tsikl uchun pul yechilishi (FULL_CYCLE yoki PARTIAL)	– (balansdan yechiladi)
LESSON_CONSUMPTION	Bitta dars iste'mol qilingani audit yozuvi (amount = 0)	0
INITIAL_BALANCE	Eski tizimdan o'tkazilayotgan o'quvchi uchun bir martalik balans	+
REFUND	Pul qaytarilgani (o'quvchi balansidan yechib chiqariladi)	– (balansdan)
SALARY_ACCRUAL	Ustoz oyligi yig'ildi (User balansiga + qarz)	+ (User balansiga)
SALARY_PAYMENT	Ustozga oylik to'landi	– (User balansidan)
EXPENSE	Markaz xarajati (ijara, ta'minot va h.k.)	– (kompaniya kassasidan)
ADJUSTMENT	Manual balans tuzatish (CEO tomonidan)	± (har ikkalasi)
TAX	Soliq (deprecated — 20260428120000 da o'chirilgan)	—

1.7 Soliq hisoblashning olib tashlanishi (muhim o'zgarish)

20260428120000_remove_tax_calculation:

Tax hisoblash logikasi tizimdan butunlay olib tashlangan. `SalaryPayment` 'dan `grossAmount`, `taxAmount`, `netAmount` maydonlari o'chirildi va o'rniga yagona `amount` qoldi. `CompanyTaxConfig` jadvali ham DROP qilindi. Hozir oylik

brutto

hisoblanadi (yig'ilgan accrual'lar yig'indisi). Soliq stavkalari faqat UI'da informatsiya sifatida ko'rsatiladi.

UI'da (Salary Breakdown Drawer'da) shunday eslatma chiqadi:

- Ustoz oyligidan — 12% (ushlanishi mumkin)
- Markaz tomonidan ustoz oyligiga — 12%
- Markaz oylik daromadidan — 4%
- Click har bir to'lovdan — 2%
- Payme har bir to'lovdan — 2%
- Uzum har bir to'lovdan — 2%

Bu raqamlar **statik infografika** — backend'da ushlanmaydi.

1.8 Partial unique indekslar (atomiklik kafolati)

```
-- Bitta davomat = bitta aktiv LESSON_CONSUMPTION
CREATE UNIQUE INDEX tx_consumption_per_attendance_unique
  ON "Transaction"("attendanceId")
  WHERE type = 'LESSON_CONSUMPTION' AND "reversedAt" IS NULL;

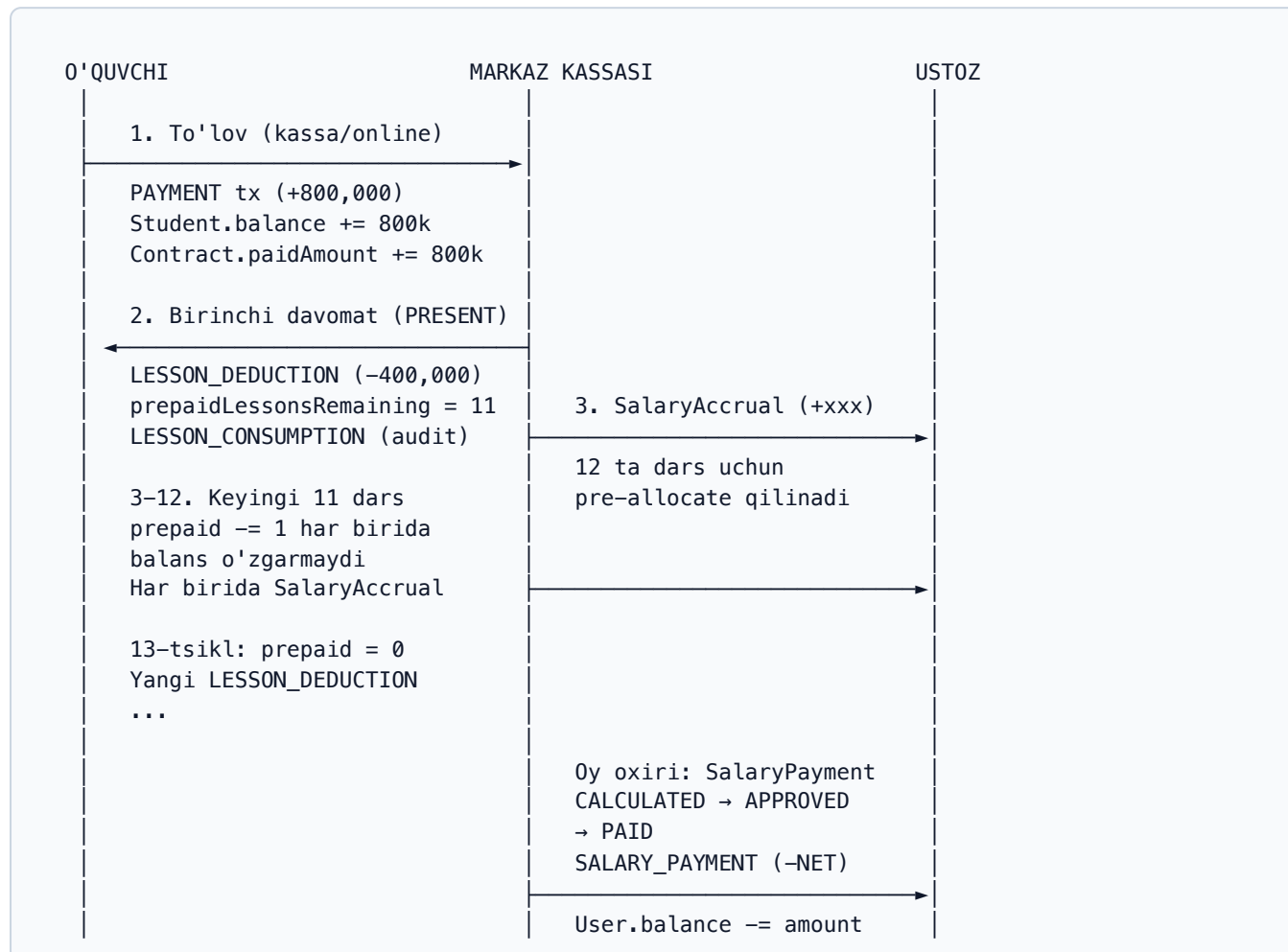
-- Bitta o'quvchi = bitta aktiv INITIAL_BALANCE
CREATE UNIQUE INDEX tx_initial_balance_per_student_unique
  ON "Transaction"("studentId")
  WHERE type = 'INITIAL_BALANCE' AND "reversedAt" IS NULL;

-- Bitta guruh + sana = bitta aktiv LessonCancellation
CREATE UNIQUE INDEX lesson_cancellation_active_unique
  ON "LessonCancellation"("groupId", "date")
  WHERE "deletedAt" IS NULL;
```

Bu indekslar reversed/soft-deleted yozuvlarni hisobga olmaydi — shuning uchun bekor qilingan operatsiyani qayta yozish mumkin (lekin asl yozuv saqlanib qoladi).

2. Pul oqimining umumiy ko'rinishi (real misollar)

2.1 Yagona pul oqimi diagrammasi



2.2 Misol 1 — Standart kurs (12 dars, 400 ming so'm)

Stsenariy:

Aliya 400,000 so'm to'ladi. Standart kurs (12 dars), per-lesson narxi: 33,333 so'm. Ustozga PERCENTAGE 30% belgilangan. Birinchi 5 dars davomida hammasini PRESENT qildi.

Vaqt	Voqea	Transaction	O'quvchi balansi	Prepaid	Ustoz accrual
T0	Aliya kassada to'laydi	PAYMENT +400,000	0 → 400,000	0	—
T1	1-dars: PRESENT	LESSON_DEDUCTION -400,000 LESSON_CONSUMPTION (0)	400,000 → 0	0 → 11	+10,000 (33,333 × 30%)
T2	2-dars: PRESENT	LESSON_CONSUMPTION (0)	0	11 → 10	+10,000
T3	3-dars: PRESENT	LESSON_CONSUMPTION (0)	0	10 → 9	+10,000
T4	4-dars: PRESENT	LESSON_CONSUMPTION (0)	0	9 → 8	+10,000
T5	5-dars: PRESENT	LESSON_CONSUMPTION (0)	0	8 → 7	+10,000

Natija: 5 dars o'tildi, ustoz uchun $5 \times 10,000 = 50,000$ so'm SalaryAccrual yig'ildi. O'quvchining balansi 0, lekin `prepaidLessonsRemaining = 7` (yana 7 dars to'langan).

2.3 Misol 2 — Intensiv kurs (20 dars, 690 ming so'm)

Stsenariy:

Bekzod 690,000 so'm to'ladi. Intensiv kurs (20 dars), per-lesson narxi: 34,500 so'm. Ustozga `FIXED_PER_STUDENT` 200,000 so'm belgilangan (butun kurs uchun, ya'ni har dars uchun = $200,000 / 20 = 10,000$ so'm).

Faza 2 audit fix:

`FIXED_PER_STUDENT` ning eski semantikasi bug edi — har darsda `value` to'liq yozilgan, natijada o'qituvchi `lessonPaymentCount` marta ortig'i bilan oylik olgan. Yangi semantika: `amount = Math.round(value / lessonPaymentCount)`. Misolda $200,000 / 20 = 10,000$ so'm har dars.

2.4 Misol 3 — Qisman to'lov (PARTIAL deduction)

Stsenariy:

Diyora 100,000 so'm to'ladi (Standart kursning yarmidan kam). Per-lesson 33,333 so'm. Birinchi davomatda nima sodir bo'ladi?

Algorithm: $\text{balans} < \text{fullCycleCost}$ (400k), lekin $\text{balans} \geq \text{perLessonCost}$ (33,333) → **PARTIAL deduction**.

```
balance = 100,000
fullCycleCost = 400,000 (yetmaydi)
perLessonCost = 33,333

lessonsCovered = floor(100,000 / 33,333) = 3 ta dars
deductionAmount = 3 × 33,333 = 99,999 so'm

→ LESSON_DEDUCTION (-99,999, mode: PARTIAL)
→ balance = 100,000 - 99,999 = 1 so'm
→ prepaidLessonsRemaining = 3 - 1 = 2 (1 ta birinchi dars uchun)
→ SalaryAccrual yoziladi (coverage = bu LESSON_DEDUCTION)
```

Keyingi 2 dars uchun ham accrual yoziladi (prepaid hisobidan). 4-darsda balans 1 so'm — yetmaydi → **B.1 qoidasi**: hech narsa yozilmaydi (no consumption, no accrual).

2.5 Misol 4 — Pul yetmasa (B.1 Coverage qoidasi)

Stsenariy:

Sardor 5,000 so'm balansi bilan keldi (per-lesson 33,333 so'm). Davomat: PRESENT.

B.1 Coverage qoidasi:

O'quvchi balansi yetmasa, pul yechilmaydi va ustozga oylik yozilmaydi. Bu —

tizimning eng muhim qoidasi

. Markaz "qarzga" dars sotmaydi: agar to'lov yo'q bo'lsa, ustozga ham hech narsa yozilmaydi.

```
balance = 5,000
perLessonCost = 33,333 (yetmaydi)

→ LESSON_DEDUCTION yo'q
→ LESSON_CONSUMPTION yo'q (audit qator ham yozilmaydi)
→ SalaryAccrual yo'q
→ Davomat status'i PRESENT, lekin moliyaviy ta'sir nol
```

Keyinroq Sardor pul to'lasa va keyingi davomat PRESENT bo'lsa, o'shanda yangi LESSON_DEDUCTION yoziladi. *Lekin o'tgan dars uchun retroaktiv accrual yozilmaydi* — **bu cheklov hozirgi kodda mavjud**.

3. Davomat va billing oqimi

3.1 Yagona giriş nuqtasi: LessonBillingService

Manual davomat (`AttendanceSaveService`) va QR davomat (`QrAttendanceScanService`) — ikkalasi ham bitta servisga delegate qiladi: `LessonBillingService.processAttendanceBilling(tx, ...)` . Bu — tizimdagi yagona pul yechish nuqtasi.

`server/src/billing/lesson-billing.service.ts`

3.2 Status transition matritsasi

Eski status	Yangi status	Harakat
—	ABSENT, EXCUSED	Hech narsa
—	PRESENT, LATE	BILL (pul yechiladi)
ABSENT, EXCUSED	ABSENT, EXCUSED	Hech narsa
ABSENT, EXCUSED	PRESENT, LATE	BILL
PRESENT, LATE	PRESENT, LATE	Hech narsa
PRESENT, LATE	ABSENT, EXCUSED	REVERSE

BILLABLE statuslar: `PRESENT` va `LATE` . Faqat shu ikkita holatda pul yechiladi.

3.3 bill() algoritmi (PRESENT/LATE statusiga o'zgarganda)

- 1 Idempotency tekshiruvi:** Shu attendance uchun aktiv (`reversedAt IS NULL`) `LESSON_CONSUMPTION` bormi? Bo'lsa, hech narsa qilmasdan chiqish (dars allaqachon to'langan).
- 2 Enrollment lock:** `SELECT * FROM Enrollment WHERE id = ? FOR UPDATE` — concurrent davomatlarni qulflash.
- 3 Guruh, kurs narxi, ustozlar topiladi:** `course.price` , `course.lessonPaymentCount` , `group.teachers` .
- 4 Strategiya tanlash:**
 - 5 (a)** `prepaidLessonsRemaining > 0` → faqat decrement, balansga tegmaydi. Coverage = eng so'nggi unreversed `LESSON_DEDUCTION` .
 - 6 (b)** `balance >= fullCycleCost` → `LESSON_DEDUCTION` (FULL_CYCLE) yoziladi, `prepaidLessonsRemaining = lessonPaymentCount - 1` . Coverage = yangi DEDUCTION.
 - 7 (c)** `perLessonCost ≤ balance < fullCycleCost` → `LESSON_DEDUCTION` (PARTIAL) yoziladi, `prepaidLessonsRemaining = floor(balance / perLessonCost) - 1` .
 - 8 (d)** `balance < perLessonCost` → **hech narsa** (B.1 qoidasi).

- 9 **LESSON_CONSUMPTION** audit qator: amount = 0, `metadata = {perLessonCost}` . Bu yozuv har bir aktiv dars uchun yagona (partial unique index orqali).
- 10 **SalaryAccrual**: har bir aktiv ustoz uchun (group.teachers), `deductionTransactionId` coverage'ga linked. Bu — **B.1 Coverage** qoidasi.

3.4 reverse() algoritmi (PRESENT/LATE → ABSENT/EXCUSED)

- 1 Aktiv `LESSON_CONSUMPTION` qidiriladi (`attendanceId` bo'yicha).
- 2 Topilsa: `reverseTransaction()` chaqiriladi → reversal qator yoziladi, `reversedAt` markeri qo'yiladi.
- 3 `prepaidLessonsRemaining += 1` — qaytarilgan darsni hisobga olish.
- 4 Har bir ustoz uchun `SalaryAccrual` qaytariladi (`reversedAt` qo'yiladi).
- 5 **Agar consumption topilmasa** (B.1 qoidasi tufayli yozilmagan edi): faqat accrual'lar reverse qilinadi (agar bo'lsa), prepaid *oshmaydi* — chunki pul yo'q edi.

3.5 Davomatni administrator yoki QR olishi farqi

Tizim `markedMethod` (MANUAL yoki QR) ni e'tiborga olmaydi. Davomatni kim qo'ygani (admin, ustoz yoki QR scan) ahamiyatga ega emas — agar status `PRESENT` / `LATE` bo'lsa, hisoblanadi va ustozga oylik yoziladi.

Foydalanuvchi savoliga javob:

"Davomat olish esidan chiqqan bo'lsa unda uni administrator olsa ham bu oylik maosh ustozga yoziladi." — Ha, bu kod darajasida tasdiqlangan. Davomatni qaysi metod bilan kim olganidan qat'iy nazar ustoz oyligi yoziladi.

4. Ustozga oylik hisoblanishi

4.1 Uch xil oylik turi

Tur	Kim uchun	Hisoblanishi	Misol
PERCENTAGE	Asosan o'qituvchilar	<code>amount = round(perLessonCost × value / 100)</code>	30%, dars 33,333 → 10,000
FIXED_PER_STUDENT	O'qituvchilar (qattiq tarif)	<code>amount = round(value / lessonPaymentCount)</code>	200,000/12 = 16,667 har dars
FIXED_MONTHLY	Admin, kassir, BD	Accrual yo'q, oy oxirida config.value to'g'ridan-to'g'ri SalaryPayment	4,000,000 so'm/oy

Muhim semantika (FIXED_PER_STUDENT):

`value` —

butun kurs (tsikl) uchun

umumiy summa, har dars uchun emas. Eski bug bo'lib, har darsda to'liq `value` yozilgan edi va o'qituvchi `lessonPaymentCount` marta ortig'i bilan oylik olgan. Faza 2 audit'da to'g'rilangan.

4.2 Per-group vs Global sozlama

`EmployeeSalaryConfig.groupId` maydoni ikki holatda bo'lishi mumkin:

- `NULL` → **Global** — bu xodimning barcha guruhleri uchun amal qiladi
- `UUID` → **Per-group** — faqat shu guruh uchun

Lookup pattern (ikki query): avval `(userId, groupId = X)` qidiriladi (per-group ustun); topilmasa `(userId, groupId IS NULL)` qidiriladi (global fallback). Postgres NULL ordering driver-specific bo'lgani uchun explicit ikki bosqich.

`server/src/salary/salary-accrual.service.ts:196-241`

4.3 Salary Config Versioning (SCD2)

Har bir oylik o'zgarishi `EmployeeSalaryConfigVersion` jadvaliga yangi qator yozadi.

`EmployeeSalaryConfig` — joriy qiymat ko'zgusi (mirror). Tarix versiyalari bo'yicha:

Misol: O'qituvchining stavkasi vaqt o'tishi bilan o'zgargan

Version 1: PERCENTAGE 30%, effectiveFrom: 2026-01-01, effectiveTo: 2026-03-15

Version 2: PERCENTAGE 35%, effectiveFrom: 2026-03-15, effectiveTo: 2026-04-20

Version 3: PERCENTAGE 40%, effectiveFrom: 2026-04-20, effectiveTo: NULL (joriy)

2026-02-10 sanasidagi dars uchun: Version 1 (30%)

2026-04-01 sanasidagi dars uchun: Version 2 (35%)

2026-04-25 sanasidagi dars uchun: Version 3 (40%)

Validatsiyalar:

- `effectiveFrom < latest.effectiveFrom` → 400 (orqaga vaqt yo'q)
- APPROVED yoki PAID `SalaryPayment` davriga tushadigan `effectiveFrom` → 400 (yopiq davrga o'zgartirib bo'lmaydi)

`server/src/salary/salary-config.service.ts:335-357`

Foydalanuvchi savoliga javob:

"Biror ustozga oylik belgilashda biz qaysi kundan boshlab buni hisoblanishini ham inobatga olishimiz kerak" — Ha, bu `effectiveFrom` orqali to'liq qo'llab-quvvatlangan. UI'da Salary Config Dialog'ida date picker bor.

4.4 Hisoblash davri (SalaryPeriodSetting)

Default: **oyning 8-sanasi**. Ya'ni davr `[oldingi oyning 8-i 00:00 Tashkent, joriy oyning 8-i 00:00 Tashkent - 1ms]`.

Sozlamalar ham SCD2 (tarixli). Faqat CEO o'zgartira oladi.

Mid-cycle cutover policy:

Agar CEO joriy davr ichida (masalan 15-aprel) yangi `cycleStartDate` belgilasa, yangi sozlama avtomatik ravishda

keyingi davr boshiga ko'chiriladi

. Joriy davr eski jadval bo'yicha yopiladi. Bu — "qaysi oy?" muammosini oldini olish uchun.

`server/src/salary/salary-period-settings.service.ts:62-83`

4.5 Cron oqimi (avtomatik hisoblash)

Tariff: `0 2 * * *` — har kuni 02:00 Tashkent vaqtida.

Logika: har bir kompaniya uchun `isCycleStartDateForCompany()` tekshiradi — agar bugungi `day-of-month` kompaniyaning `cycleStartDate` 'ga teng bo'lsa, oylik hisoblash boshlanadi.

`server/src/salary/salary-cron.service.ts:25-62`

4.6 calculateMonthlySalaries() — to'liq oqim

A) Accrual-based (PERCENTAGE / FIXED_PER_STUDENT)

- 1 Davr aniqlash: faol `SalaryPeriodSetting` topiladi → `[periodStart, periodEnd]`
- 2 Unpaid accruallarni yig'ish: `WHERE salaryPaymentId IS NULL AND reversedAt IS NULL AND lessonDate BETWEEN periodStart AND periodEnd`
- 3 Userlar bo'yicha guruhlash: `Map<userId, {ids[], total}>`
- 4 Atomik blok har bir o'qituvchi uchun:
 - 5 `SalaryPayment` CREATE (status: `CALCULATED`, amount = total)
 - 6 Yig'ilgan accrual'larni `salaryPaymentId` bilan link qilish
 - 7 `applyPendingAdvances()` — TEACHER_ADVANCE settlement

B) Fixed-monthly (admin, kassir, BD)

- 1 Parent `EmployeeSalaryConfig` ni topish (`salaryType = FIXED_MONTHLY, isActive = true, groupId IS NULL`)
- 2 periodEnd sanasida faol `EmployeeSalaryConfigVersion` ni o'qish (future-dated rate change'lar joriy davrga ta'sir qilmaydi)
- 3 Idempotency — bu davr uchun allaqachon payment bo'lsa, skip
- 4 `SalaryPayment` yaratish, advance settlement

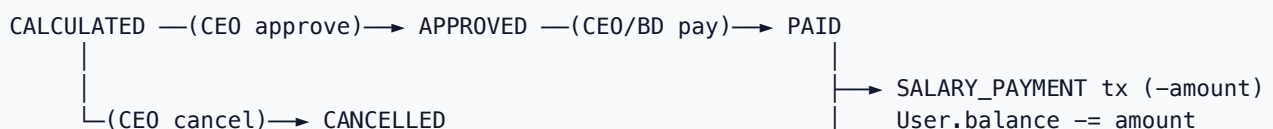
4.7 TEACHER_ADVANCE settlement (avans ushlab qolish)

Agar markaz oy davomida o'qituvchiga avans bergan bo'lsa (`Expense.category = TEACHER_ADVANCE, relatedUserId = teacherId, settledBySalaryPaymentId IS NULL`), oylik hisoblanganda bu avanslar netto summadan ushlanib qolinadi.

Tartib: `createdAt` bo'yicha (eski avans avval). Agar bitta avans qoplay olmaydigan miqdorda bo'lsa, qolgan avans keyingi oyga o'tadi.

server/src/salary/salary-calculation.service.ts:198–236

4.8 Status oqimi



4.9 Salary Breakdown — har dars qayerdan kelgani

`SalaryBreakdownService` (*server/src/salary/salary-breakdown.service.ts:79–164*) per-payment yoki per-cycle tahlil qaytaradi:

Maydon	Mazmuni
lessonDate	Dars sanasi
student	O'quvchi (ism, ID)
group	Guruh (ism, kurs)
perLessonCost	Dars narxi (snapshot — kurs narxi keyinroq o'zgarsa ham bu yerda saqlanadi)
amount	Hisoblangan summa
salaryConfigVersion	Qaysi versiya bo'yicha hisoblangan (salaryType, value, scope)
scope	GROUP (per-group config) yoki GLOBAL
reversedAt / reversalReason	Bekor qilingan accrual'lar uchun

Totallar: active accrual count, active total, reversed count, reversed total.

Foydalanuvchi savoliga javob:

"Ustoz oyligidan aniq har bir summa qayerdan kelgani nega bunday summaga kelib qolgani haqidagi to'liq hisobotni ko'ra olishimiz shart" — Ha, Salary Breakdown Drawer (UI) va **GET** [/salary/payments/:id/breakdown](#) endpoint orqali har dars uchun: o'quvchi, guruh, sana, dars narxi, qaysi stavka qo'llangan, qaysi versiya — barchasi ko'rinadi. CSV eksport ham bor.

4.10 Teacher Timeline (taymlayn)

TeacherTimelineService uch turdagi event'larni birlashtiradi va xronologik feed qaytaradi:

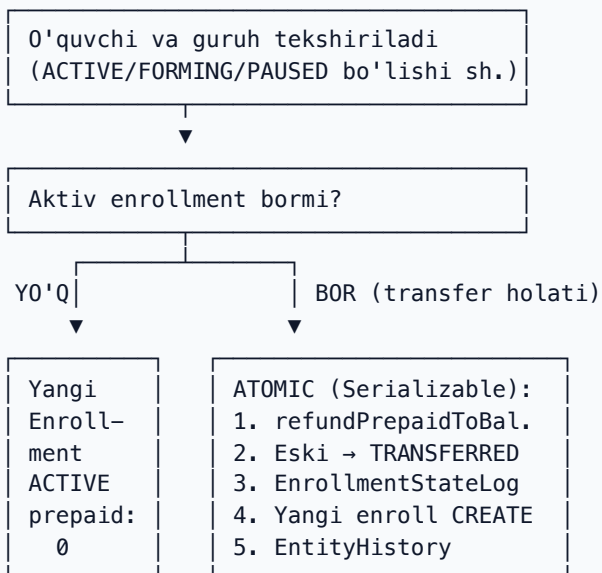
Event turi	Manba	Misol
SalaryChanged	EmployeeSalaryConfigVersion	"Stavka 30% → 35% o'zgartirildi"
GroupAdded	GroupTeacherHistory	"A-101 guruhiga qo'shildi"
GroupRemoved	GroupTeacherHistory	"B-202 guruhidan chiqarildi"
GroupReplaced	GroupTeacherHistory	"C-303 → D-404 ga o'tkazildi"
ProfileUpdated	EntityHistory (User)	Profil ma'lumotlari yangilandi

Endpoint: **GET** [/salary/timeline/:userId](#) — CEO, BD, Admin uchun.

5. O'quvchi hayot tsikli moliyaviy nuqtai nazardan

5.1 O'quvchini guruhga qo'shish (enroll)

Endpoint: `POST /students/:id/enroll` — CEO, BD, Admin



Yangi enrollment dastlab `prepaidLessonsRemaining = 0` bilan boshlanadi. Birinchi PRESENT/LATE davomatda balans asosida deduction yoziladi (B.1 qoidasi bilan).

Transfer holida: agar o'quvchi yangi guruhda boshqa ustoz bo'lsa, `transferReasonId` majburiy bo'lishi mumkin.

server/src/students/student-enrollment.service.ts:24-274

5.2 O'quvchini guruhdan chiqarish (DROPPED / removeFromGroup)

Endpoint: `DELETE /students/:id/enroll/:enrollmentId`

Asosiy ish: qolgan prepaid darslarning pulini o'quvchi balansiga qaytarish.

Hisoblash formulasi:

`perLessonCost` = oxirgi unreversed `LESSON_DEDUCTION` dan
`metadata.perLessonCost`
(kurs narxi keyin o'zgargan bo'lsa ham
asl narx qo'llanadi)

`refundAmount` = `prepaidLessonsRemaining` × `perLessonCost`

ATOMIC TRANSACTION (Serializable):

1. `refundPrepaidToBalance`:
ADJUSTMENT TX (+`refundAmount`)
`Student.balance` += `refundAmount`
`prepaidLessonsRemaining` = 0
2. `Enrollment.status` = DROPPED
`departureReasonId` set
`statusChangeReason` set
`EnrollmentStateLog` yoziladi
3. `EntityHistory`: DROPPED

Foydalanuvchi savoliga javob:

"O'quvchi krus uchun to'lov qildi va o'qishini bir muddatga muzlatdi" — Bu holat to'liq hal qilingan. O'quvchi pul to'lasa, lekin davomatga qatnashmasa, balans yechilmaydi (B.1). Davomatga qatnashganda — o'shanda yechiladi. Agar guruhdan butunlay chiqsa, qolgan darslarning puli balansga qaytariladi.

5.3 Guruhdan guruhga ko'chirish (TRANSFERRED)

Yuqorida tasvirlangan *transfer* holati. Bu — o'quvchini bir guruhdan boshqasiga ko'chirishda **atomik**:

- 1 Eski enrollmentdagi qolgan prepaid darslarning puli balansga qaytariladi
- 2 Eski enrollment **TRANSFERRED** statusiga o'tadi
- 3 Yangi enrollment yaratiladi (`prepaidLessonsRemaining` = 0)
- 4 Yangi guruhda birinchi davomatda yangi balans asosida deduction yoziladi

Foydalanuvchi savoliga javob:

"Bir o'quvchini bugun A guruhdan keyin esa B guruhda o'qisa A guruhda o'qiganida uni to'lovidan A guruh ustoziga qancha olishi kerak bo'lsa shuncha oladi (1 ta dars uchun hisoblab). Keyin esa yangi guruhda o'qiganida undan B guruhi ustoziga oladi." — Ha, bu to'liq qo'llab-quvvatlangan:

- A guruhdagi har dars uchun A ustoziga accrual yozilgan (transfer payti reverse qilinmaydi)
- Transfer paytida qolgan prepaid pul balansga qaytariladi
- B guruhda yangi deduction'dan B ustoziga accrual yoziladi

5.4 Status o'zgarishi: ACTIVE / FROZEN / DROPPED / GRADUATED

Endpoint: `PATCH /students/:id/status`

Statuslar:

Status	Mazmuni	Pul oqimi ta'siri
ACTIVE	Faol o'qiyapti	Davomatda yechiladi
FROZEN	Muzlatilgan (vaqtinchalik)	Davomatga qatnashmasa, hech narsa yechilmaydi (B.1)
DROPPED	Tark etgan	refundPrepaidToBalance — qolgan pul qaytariladi
GRADUATED	Bitirgan	Avtomatik (manual reject qilinadi)

Cascade: Student status o'zgarsa, `StatusCascadeService` aktiv enrollmentlarni mos statusga o'tkazadi.

Status sababi (ExitReason): O'zgarish sababini tanlash kerak. `StudentExitReason` jadvalida sabablar katalogi mavjud (CEO/BD/Admin tomonidan boshqariladi). Har sababga `appliesTo` array — qaysi exit turlari uchun ishlatish mumkinligi.

ExitType	Qachon
GROUP_REMOVAL	Guruhdan chiqarish (enrollment level)
FREEZE	Muzlatish (FROZEN status)
EXPEL	Chetlatish (DROPPED, jiddiy sabab)
INACTIVE	Nofaol bo'lib qolish
ARCHIVE	Arxivga olish

Migration 20260429114132:

Eski `EnrollmentDepartureReason` jadvali `StudentExitReason` ga unifikatsiya qilingan. Endi bitta katalog 5 turdagi exit uchun ishlatiladi. Eski `departure-reasons` moduli o'chirilgan.

5.5 O'quvchini muzlatish (FROZEN) — to'liq oqim

Stsenariy:

Aliya 15-aprelda 500,000 so'm to'ladi va 1-maygacha o'qishini muzlatdi. 1-mayda u davomatga qatnashdi.

Pul oqimi:

Sana	Voqea	Balans	Prepaid
15-apr	To'lov 500k	0 → 500,000	0
15-apr	FROZEN qilindi	500,000	0 (davomatga kelmadi)
16–30 apr	Davomat yo'q	500,000	0 (yechilmaydi)
1-may	ACTIVE qilindi, PRESENT	500,000 → ?	?

1-mayda PRESENT bo'lganida birinchi davomat sifatida bill() ishlaydi:

- $\text{balance} = 500,000 \geq \text{fullCycleCost} (400k) \rightarrow \text{FULL_CYCLE deduction}$
- $\text{balance} \rightarrow 100,000$
- $\text{prepaidLessonsRemaining} = 11$
- SalaryAccrual yoziladi (1-may sanasi va o'sha kunda faol bo'lgan ustozlar uchun)

Foydalanuvchi savoliga javob:

"O'quvchi qachonki davomatda qatnashsa oyligi shundan boshlab hisoblanadi" — Ha, bu to'liq to'g'ri ishlaydi. Tizim — pull-based: faqat davomat hodisasi bo'lganida yechiladi va yoziladi.

5.6 Guruh ichida ustoz o'zgarishi

Agar guruh A da Botir va Sardor edi, endi Sardor o'rniga Farxod qilinsa:

- 1 **GroupTeacher** jadvalida DELETE (Sardor) + INSERT (Farxod)
- 2 **GroupTeacherHistory** yoziladi: `changeType = REPLACED, previousTeacherIds = [Sardor], newTeacherIds = [Farxod]`
- 3 **O'tgan darslar uchun:** Sardor allaqachon olgan accruallarini saqlab qoladi (reverse qilinmaydi)
- 4 **Kelajakdagi darslar uchun:** Botir va Farxod accrual oladi, Sardor — yo'q

Foydalanuvchi savoliga javob:

"Guruh ma'lum bir vaqt A ustozda o'qib keyin B ustozga o'tishi" — Ha. Dars sanasidagi *aktiv* ustozlar accrual oladi. Eski ustozning o'tgan darslari saqlanadi.

6. Pul qaytarish (refund) va to'lov bekor qilish (reverse)

6.1 Refund eligibility (qaytarilishi mumkinmi?)

Holat	Qaytarish miqdori
Kurs hali boshlanmagan (<code>startDate > now</code>)	100% qaytariladi (oldingi refundlar minus qilinadi)
50% va ortiq dars o'tilgan	0% — refund yo'q (BadRequestException)
50% dan kam dars o'tilgan	<code>paidAmount - consumedFromLedger - previousRefunds</code>

consumedFromLedger formulasi:

```
consumedFromLedger = ABS(SUM(LESSON_DEDUCTION
                             WHERE reversedAt IS NULL
                             AND reversedTransactionId IS NULL))
```

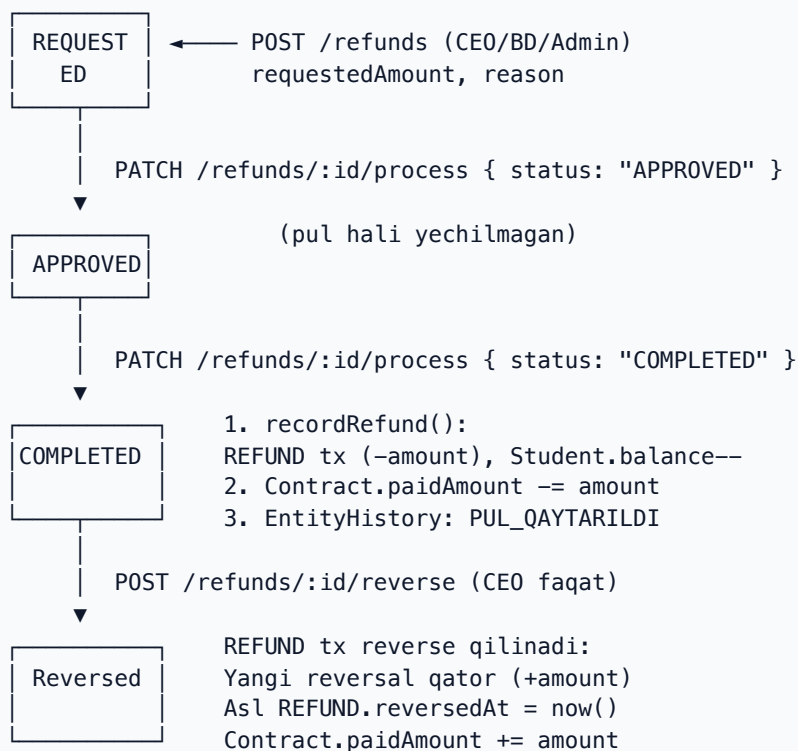
Ikkita filter birgalikda ishlatiladi:

- `reversedTransactionId: null` — bu yozuv reversal entry emas
- `reversedAt: null` — bu yozuv o'ziga reversed bo'lmadi

Bu filterlar oldingi double-counting bug'ini hal qilgan: agar admin 100k yechib, keyin dars bekor qilingan (80k teskari) bo'lsa, hozir refund'ga 20k hisoblanadi (eski bug'da 100k bolardi).

server/src/refunds/refunds-create.service.ts:88-141

6.2 Refund jarayoni



6.3 Payment reverse (CEO faqat)

Endpoint: `POST /payments/:id/reverse`

Yangi muhim guard (Faza 8):

Block:

Agar shu to'lovdan keyin (`createdAt > payment.createdAt`) o'quvchi nomidan aktiv (`reversedAt IS NULL`) `LESSON_CONSUMPTION` mavjud bo'lsa — to'lovni reverse qilib bo'lmaydi. Server 400 qaytaradi: "Pul allaqachon darslarga sarflangan".

Idempotency: agar `payment.status === REVERSED` bo'lsa, ikkinchi marta chaqirish 400 ga yo'lliqadi.

Atomik oqim:

- 1 Payment topiladi va status tekshiriladi
- 2 PAYMENT transaction topiladi (`reversedAt IS NULL`)
- 3 Downstream `LESSON_CONSUMPTION` tekshiruvi (yuqoridagi guard)
- 4 `reverseTransaction()` — reversal qator yoziladi, asl `reversedAt` markeri qo'yiladi
- 5 `Student.balance -= amount` (pul olib tashlanadi)
- 6 `Contract.paidAmount -= amount`
- 7 `Payment.status = REVERSED`
- 8 EntityHistory: `TO'LOV_BEKOR_QILINDI`

6.4 Lesson Deduction Reverse

Endpoint: `POST /billing/lesson-deduction/:id/reverse` — CEO, BD

Maqsad: butun batch (12 yoki 20 ta dars uchun yechilgan pul)ni qaytarish.

Misol: admin noto'g'ri 400k yechilgan
→ `POST /billing/lesson-deduction/:id/reverse`

ATOMIK:

1. `LESSON_DEDUCTION` reverse (balance +400k)
2. Linked `SalaryAccrual`'lar reverse (deductionTransactionId orqali)
3. `LESSON_CONSUMPTION`'lar reverse (createdAt >= deduction.createdAt)
4. `Enrollment.prepaidLessonsRemaining` = 0

Natija:

- O'quvchi balansi qaytariladi
- Ustozlar olgan accruallar bekor qilinadi
- Hisoblar restart bo'ladi (yangi davomat → yangi deduction)

7. Dars bekor qilish (lesson cancellation) va dam olish kunlari

7.1 Lesson Cancellation Cascade

Endpoint: `POST /lesson-cancellations` — CEO, BD, Admin

Bu — bitta guruhning bitta sanasini bekor qilish (masalan, ustoz kasal bo'lib qoldi). Holiday'dan farqi: faqat bir guruh uchun.

Atomik oqim:

- 1 Guruh tekshiriladi va bu sana uchun aktiv cancellation bormi (partial unique index)
- 2 `LessonCancellation` qator yoziladi
- 3 Shu kun va guruhdagi PRESENT/LATE attendance'lar topiladi
- 4 Har biri uchun:
 - 5 Status: `EXCUSED` ga o'zgartiriladi
 - 6 `cancellationId` set qilinadi
 - 7 `processAttendanceBilling(oldStatus=PRESENT, newStatus=EXCUSED)` ishga tushadi → reverse path
 - 8 LESSON_CONSUMPTION reverse, prepaid +1, SalaryAccrual reverse
- 9 EntityHistory: DARS_BEKOR_QILINDI

7.2 Cancellation soft-delete (qayta tiklash)

Endpoint: `DELETE /lesson-cancellations/:id` — CEO, BD

Muhim cheklov:

Soft-delete

attendance va billing'ni avtomatik tiklamaydi

. `deletedAt` markeri qo'yiladi, lekin EXCUSED'ga o'zgartirilgan davomatlar EXCUSED bo'lib qoladi. Admin keyin qo'lda PRESENT bo'yicha qayta davomat olishi kerak.

UI'da bu haqida ogohlantirish chiqadi: "Diqqat: bu davomat va to'lovni tiklamaydi."

7.3 Holiday (kompaniya bo'yicha)

Holiday — butun kompaniya/filial bo'yicha bekor qilingan kun (masalan, davlat bayrami). Bu Holiday jadvalida boshqariladi va davomat formasi shu sanada PRESENT'ni rad qiladi.

Holiday yaratilsa, o'sha sanada davomat olib bo'lmaydi — shuning uchun pul yechilmaydi va ustozga oylik yozilmaydi (B.1 emas, balki davomat hodisasi yo'qligi).

Foydalanuvchi savoliga javob:

"Dam olish kunlari, guruhning ma'lum sanadagi darsi bo'lmashini belgilash" — Holiday va LessonCancellation orqali to'liq qo'llab-quvvatlangan. Pul yechilmaydi va ustoz oyligi yozilmaydi.

8. Initial Balance va eski tizimdan migratsiya

8.1 Foydalanuvchi muammosi

Foydalanuvchi yozgan: "Hozir o'quv markazi avvaldan faoliyat olib boryotgan markaz hisoblanadi. Oldin tizim bo'lmagani uchun kim qachon to'lov qilgani unchalik ham aniq emas. Markaz to'lovlarni oylik tizimda qabul qilgan. Har oyning 1 sanasida misol uchun. Shunga biz buni to'liq tizimga o'tkazishimiz uchun ya'ni 12 talik yoki 20 talik (kurs sonidan kelib chiqib) qanday o'tkazishimiz mumkin."

8.2 Yechim — POST /students/:id/initial-balance

Endpoint: CEO faqat. DTO:

```
{
  "amount": 500000,    // 0+ (manfiy emas)
  "note": "Eski tizimdan o'tkazildi"
}
```

Oqim:

- 1 O'quvchi topiladi (multi-tenant filter)
- 2 Balans qulflanadi (`SELECT FOR UPDATE`)
- 3 `INITIAL_BALANCE` Transaction yoziladi (amount = +500,000)
- 4 `Student.balance += amount`
- 5 Partial unique indeks qo'llanadi: bitta o'quvchi uchun yagona aktiv (`reversedAt IS NULL`)
`INITIAL_BALANCE`
- 6 EntityHistory: BOSHLANG'ICH_BALANS

8.3 Tavsiya etilgan ish jarayoni (workflow)

Misol stsenariy:

Eski tizimda Aliya 1-sanada 400,000 so'm to'lab, 12-talik standart kursga 8 ta dars borgan. Endi yangi tizimga o'tkazish kerak.

Qadam	Harakat	Natija
1	Aliyani tizimga qo'shish (Student CREATE)	balance = 0, status = ACTIVE
2	Guruhga enroll qilish	Enrollment ACTIVE, prepaid = 0
3	Eski tizimdagi qoldiqni hisoblash: $qoldiq = 400,000 - (8 \times 33,333) \approx 133,336$	—
4	<code>POST /students/:id/initial-balance</code> { amount: 133,336 }	INITIAL_BALANCE +133k, balance = 133,336
5	Yangi davomat boshlanganda PARTIAL deduction yoziladi: $\text{floor}(133,336 / 33,333) = 4$ ta dars	balance = ~3, prepaid = 3

8.4 Initial Balance reversibility

Agar admin xato qilgan bo'lsa va INITIAL_BALANCE'ni qayta yozish kerak bo'lsa:

- 1 `POST /transactions/:id/reverse` — reversal qator yoziladi, asl `reversedAt` markerlanadi
- 2 Endi yana `POST /students/:id/initial-balance` chaqirish mumkin (eski reversed bo'lgani uchun unique constraint o'tib bo'ladi)

Muhim:

Agar INITIAL_BALANCE allaqachon LESSON_CONSUMPTION uchun ishlatilgan bo'lsa, payment reverse'dagi guard'ga o'xshash himoya hozircha INITIAL_BALANCE uchun *aniq qilib qo'ymagan*. CEO qo'lda tekshirishi kerak.

9. Markaz xarajatlari va TEACHER_ADVANCE

9.1 Xarajat kategoriyalari

Kategoriya	Mazmuni
RENT	Ijara haqi
UTILITIES	Kommunal (svet, suv, internet)
SUPPLIES	Ofis ta'minoti
MARKETING	Reklama xarajatlari
TEACHER_ADVANCE	O'qituvchi avansi (oylik hisoblanganda ushlab qolinadi)
OTHER	Boshqa

9.2 TEACHER_ADVANCE oqimi

AVANS BERISH:

```
POST /expenses { category: "TEACHER_ADVANCE", relatedUserId: 42, amount: 200000 }
```

↓

- Expense yaratiladi (settledBySalaryPaymentId = NULL)
- EXPENSE Transaction yoziladi (markaz kassasidan)

OY OXIRIDA (calculateMonthlySalaries):

↓

applyPendingAdvances(userId = 42, salaryPayment) chaqiriladi:

- WHERE relatedUserId = 42 AND settledBySalaryPaymentId IS NULL
- ORDER BY createdAt ASC (eski avans avval)
- Har bir avans uchun:
 - * Agar deducted + advance.amount <= salaryPayment.amount → settle
 - * settledBySalaryPaymentId = salaryPayment.id
 - * deducted += advance.amount
 - * Aks holda: keyingi oyga qoldiriladi

natija:

finalNet = salaryPayment.amount - deducted

SalaryPayment yangilanadi

9.3 Xarajat tahrirlash va o'chirish

- **PATCH /expenses/:id** — moliyaviy maydon o'zgarsa, eski Transaction reverse qilinadi va yangisi yoziladi
- **DELETE /expenses/:id** — soft delete (deletedAt) + ledger reversal

10. Maxsus holatlar (edge cases)

10.1 Pul yetmasa nima bo'ladi (B.1 qoidasi)

Asosiy qoida:

O'quvchi balansi `perLessonCost` dan kam bo'lsa, davomat PRESENT bo'lsa ham:

- LESSON_DEDUCTION yozilmaydi
- LESSON_CONSUMPTION yozilmaydi (audit qator ham yo'q)
- SalaryAccrual yozilmaydi
- Davomat status'i qoladi (PRESENT), lekin moliyaviy ta'sir nol

Bu — markaz "qarzga sotmasligi"ning kafolati. Lekin shuni e'tiborga olish kerak:

keyin pul to'lansa, o'tgan dars uchun retroaktiv accrual yozilmaydi

. Bu — joriy kodning chegarasi.

10.2 Davomat keyinroq olinsa

Stsenariy: ustoz davomat olishni unutdi, keyin admin 2 kundan keyin oladi. PRESENT qo'yilsa:

- O'sha vaqtda balans yetadigan bo'lsa → pul yechiladi va accrual yoziladi
- Accrual `lessonDate` = asl dars sanasi (admin qo'yganida emas)
- Salary versiya lookup ham asl `lessonDate` bo'yicha qilinadi

10.3 Ko'p marta to'lov qilinishi

Stsenariy: o'quvchi 3 marta 100k to'laydi.

- Har biri alohida PAYMENT tx + balans qulflash
- balanceBefore/After lar zanjirli: [0→100, 100→200, 200→300]
- Student.balance = 300,000
- Bir to'lov reverse qilinsa, faqat shu to'lov ta'sir qiladi (ledger zanjir uzilishi mumkin lekin *reversedAt* bilan filtr qilinadi)

10.4 Refund qilingan to'lovni reverse qilish

Refund — bu yangi REFUND transaction. Asl PAYMENT'ni reverse qilish guard'i: agar shu to'lovdan keyin LESSON_CONSUMPTION bo'lsa — block. Refund consumption emas, shuning uchun bu guard ishlamasligi mumkin.

E'tibor talab qiluvchi joy:

Refund qilingan to'lovni ham reverse qilish guarid'i hozirgi kodda aniq emas. Bu test qilinishi kerak holatlardan biri.

10.5 Concurrent davomat (race condition)

Stsenariy: ikkita admin bir vaqtning o'zida bir o'quvchi uchun davomat oladi.

- `SELECT FOR UPDATE` Enrollment'ni qulflaydi
- Birinchi tranzaksiya yakunlanmaguncha ikkinchisi kutadi
- Partial unique indeks (consumption per attendance) ikkinchi yozuvni rad qiladi
- Natija: idempotent — bir attendance uchun bir LESSON_CONSUMPTION

10.6 Cycle bo'yicha multiple deductions

Stsenariy: o'quvchi 12 dars uchun pul yechilgan (1-tsikl), 12 dars o'tgan, prepaid = 0. 13-darsda balans yana yetadi → 2-tsikl uchun yangi LESSON_DEDUCTION.

- Har tsikl alohida LESSON_DEDUCTION (har biri o'z metadata bilan)
- SalaryAccrual'lar har biri o'z `deductionTransactionId` coverage'ga bog'lanadi
- Salary breakdown'da har dars qaysi tsikldan ekanligini ko'rish mumkin

10.7 Ustoz almashtirilgandan keyin reverse

Stsenariy: 10-marsda Sardor 5 ta dars uchun accrual oldi. 12-marsda Farxod o'rnatildi. 11-mars sanasidagi davomat ABSENT'ga o'zgartirilsa:

- Sardor uchun yozilgan SalaryAccrual reverse qilinadi (`reversedAt`)
- Yangi accrual yozilmaydi (chunki davomat ABSENT)
- Salary breakdown'da reversed qator alohida ko'rinadi

10.8 FIXED_MONTHLY ustozning past dare's

FIXED_MONTHLY ustoz davomat olmasin yoki olsin — oylik value to'g'ridan-to'g'ri SalaryPayment ga aylanadi. Bu administratorlar uchun qulay.

Edge case:

Agar FIXED_MONTHLY ustoz oy o'rtasida ishga kirgan/ishdan ketgan bo'lsa, hozir tizim *to'liq oylik* yozadi. Pro-rata hisoblash hozircha yo'q.

11. Frontend UI yo'llari va sahifalari

11.1 /payments/salary — Oylik ro'yxati (CEO/BD/Admin/Cashier)

Asosiy moliyaviy boshqaruv sahifasi:

- **Jadval:** # · Xodim · Lavozim · Davr · Summa · Holat · Amallar
- **Status badge:** Hisoblangan (amber), Tasdiqlangan (blue), To'langan (green), Bekor qilingan (red)
- **Amallar:**
 - CALCULATED → "Tasdiqlash" tugmasi
 - APPROVED → "To'lash" tugmasi
 - PAID → to'langan sana
- **Yuqori panel (CEO faqat):**
 - "Davрни yakunlash" — POST /salary/calculate (qo'lda trigger)
 - "Sozlamalar" dropdown:
 - Xodim stavkalari → SalaryConfigDialog
 - Hisoblash davri → SalaryPeriodSettingsSheet
 - "Hammasini to'lash" — batch payment
- **Qatorga bosish:** SalaryBreakdownDrawer ochilib, har dars qayerdan kelgani ko'rinadi (CSV eksport bilan)

11.2 SalaryConfigDialog — Xodim stavkalarini belgilash

Form maydonlari:

- Xodim tanlash (rolga qarab forma o'zgaradi)
- Oylik turi:
 - Barcha xodimlar uchun: FIXED_MONTHLY
 - Faqat o'qituvchi uchun: PERCENTAGE va FIXED_PER_STUDENT ham
- Qiymat (PriceInput, % yoki so'm)
- **Kuchga kirish sanasi (effectiveFrom)** — DatePicker, ixtiyoriy
- Eslatma: "Yangi qoida shu sana va undan keyingi darslarga qo'llanadi"

Mavjud konfiguratsiya bo'lsa, "Joriy sozlama:" qutida ko'rsatiladi.

11.3 SalaryBreakdownDrawer

Layout:

- Header: "Oylik tafsilotlari" + status badge
- Summary card: summa, davr, dars soni, o'quvchi soni, bekor qilinganlar
- Per-dars jadvali (5 ustun): # · Sana · O'quvchi/Guruh · Stavka qoidasi · Summa

- Reversed qatorlar: opacity 60%, strikethrough
- CSV download: `oylik-tafsilotlari-{paymentId}.csv`
- Pastida: PossibleDeductionsInfo (ushlanishi mumkin bo'lgan deductionlar info'si)

11.4 SalaryPeriodSettingsSheet

- Joriy davr ko'rsatiladi: "Har oyning 8-kunidan, 01.01.2026 dan kuchda"
- Yangi sozlama:
 - Boshlanish kuni (1-28)
 - Kuchga kirish sanasi
- Tarix jadvali — barcha versiyalar (faol/yopilgan)

11.5 /profile/salary — Ustozning o'z oyligini ko'rishi

Faqat o'qituvchilar uchun (lehrer portal):

- 3 ta KPI karta:
 - Kutilayotgan oylik (joriy davr forecast'i)
 - Real ishlangan (yig'ilgan accruallar)
 - To'langan (oxirgi PAID payment)
- Guruhlar bo'yicha (PERCENTAGE/FIXED_PER_STUDENT uchun):
 - Aktiv o'quvchilar soni
 - Oylik darslar soni
 - Stavka
 - $\text{Kutilayotgan oylik} = \text{activeStudents} \times \text{lessonsPerMonth} \times \text{rate}$
- Joriy davr — har dars tafsilot
- PossibleDeductionsInfo (variant=teacher) — faqat 12% ushlanish

Foydalanuvchi savoliga javob:

"Ustozlar o'z profilida oylik maoshlarlarini kuzatib borishi mumkin. Ulardagi aktiv o'quvchilar sonidan kelib chiqqan holda qancha oylik maosh kutishlari mumkinligi va qilingan to'lovlar asosida ularda real oylik maosh qancha ekanligini ko'rishlari mumkin bo'lishi kerak." — Bu to'liq amalga oshirilgan: [/profile/salary](#) sahifasi va `GET /salary/me/summary` endpoint kutilayotgan vs haqiqiy oylikni ko'rsatadi.

11.6 Teacher Profile — "Ish haqi" va "Taymlayn" tablari

Tab	Kim ko'radi	Mazmuni
Guruhlar	Hammasi	Ustozning guruhlari
Izohlar	Hammasi	Comment thread
Tarix	Hammasi	EntityHistory
Taymlayn	CEO/BD/Admin	Salary changes + group history + profile updates
Ish haqi	CEO/BD	4 ta KPI + per-group breakdown

11.7 Student Profile — yangi tablar

Tab	Endpoint	Mazmuni
To'lovlar	GET /transactions/student/:id? types=PAYMENT,REFUND,ADJUSTMENT,INITIAL_BALANCE	Pul oqimi
Darslar (Lesson Trail)	GET /transactions/student/:id/lesson-trail	LESSON_DEDUCTION + LESSON_CONSUMPTION

Lesson Trail — har dars qayerdan kelgani va qaysi tsiklga kirganligini paginated jadval ko'rinishida.

Initial Balance Dialog (CEO faqat) — student profile dropdown menyusidan ochiladi.

11.8 Group Detail — "Bekor qilingan" tab (yangi)

- "Bu darsni bekor qilish" tugmasi (CEO/BD/Admin)
- Cancellations jadvali: # · Sana · Sabab · Bekor qilgan · Vaqti · [Delete]
- Delete uchun confirmation dialog: "Diqqat: bu davomat va to'lovni tiklamaydi"

11.9 Departure Reasons Manager

Reports → Departed Students sahifasidagi "Ketish sabablari" tugmasi orqali ochiladi:

- Yangi sabab qo'shish:
 - Sabab nomi
 - Qaysi exit turlariga qo'llanishi (5 ta checkbox: GROUP_REMOVAL, FREEZE, EXPEL, INACTIVE, ARCHIVE)
- Mavjud sabablarni tahrirlash, o'chirish (soft delete)

11.10 Navigation o'zgarishlari

- Yangi: `/profile/salary` — faqat o'qituvchilar (visibleForRoles: [4])
- Settings nav'dan "Tark etgan o'quvchilar" o'chirilgan (Reports'ga ko'chgan)

12. Rollar va ruxsatlar matritsasi

Operatsiya	CEO	BD	Admin	Cashier	Teacher
To'lov yaratish (POST /payments)	✓	✓	✓	✓	—
To'lov bekor qilish (reverse)	✓	—	—	—	—
Ledger ko'rish (transactions)	✓	✓	—	—	—
Lesson trail (per-student)	✓	✓	✓	✓	—
Initial Balance qo'shish	✓	—	—	—	—
Manual ADJUSTMENT	✓	✓	—	—	—
Refund yaratish	✓	✓	✓	—	—
Refund tasdiqlash/to'lash	✓	✓	—	—	—
Refund bekor qilish	✓	—	—	—	—
Salary config CRUD	✓	—	—	—	—
Salary period sozlash	✓	—	—	—	—
Salary calculate (manual)	✓	—	—	—	—
Salary approve	✓	—	—	—	—
Salary pay (single, batch)	✓	✓	—	—	—
Salary breakdown (boshqa ustoz)	✓	✓	✓	—	—
Lesson Deduction reverse	✓	✓	—	—	—
Lesson Cancellation create	✓	✓	✓	—	Faqat o'z guruhi
Lesson Cancellation delete	✓	✓	—	—	—
Expense CRUD	✓	✓	create/list	—	—
Reports (financial)	✓	✓	—	—	—
O'z oyligini ko'rish (/salary/me/*)	—	—	—	—	✓
Student status change	✓	✓	✓	—	—
Student exit reasons CRUD	✓	✓	—	—	—

Foydalanuvchi savoliga javob:

"Ustozlarga oylik belgilash faqat CEO tomonidan UX tomonlama ham backend tomonlama ham bu boshqalarga ko'rinmasligi va ruxsat berilmasligi shart" — Bu to'liq amalga oshirilgan. Backend'da `@Roles('CEO')` guard bilan, frontend'da `isCeo` shart bilan UI yashiriladi. BD/Admin to'g'ridan-to'g'ri API'ga so'rov yuborsa 403 oladi.

13. Topilgan kamchiliklar va e'tibor talab qiluvchi joylar

13.1 Retroaktiv accrual yo'qligi

Holat:

O'quvchi balansi yetmasligi sababli dars uchun pul yechilmagan bo'lsa (B.1 qoidasi), keyin pul to'lansa *o'tgan dars uchun* retroaktiv accrual yozilmaydi.

Tavsiya:

CEO bilan muhokama qilib qaror qabul qilish — *retroaktiv accrual* kerakmi yoki yo'q.

13.2 FIXED_MONTHLY pro-rata yo'qligi

Holat:

FIXED_MONTHLY xodim oy o'rtasida ishga kirgan/ketgan bo'lsa ham, oylik to'liq yoziladi.

Tavsiya:

agar real ishga kirish/chiqish kuni asosida pro-rata kerak bo'lsa, `EmployeeSalaryConfig.startDate` va `endDate` qo'shish kerak.

13.3 Refund qilingan to'lovni reverse qilish guard

Holat:

Payment reverse'da LESSON_CONSUMPTION guard bor, lekin REFUND transaction guard'i aniq emas.

Tavsiya:

agar to'lov uchun refund qilingan bo'lsa, payment'ni reverse qilish hech bo'lmaganda ogohlantirish bilan amalga oshirilishi kerak.

13.4 Lesson Cancellation soft-delete qaytarish

Holat:

LessonCancellation o'chirilsa, EXCUSED'ga o'zgartirilgan davomatlar tiklanmaydi.

Tavsiya:

bu yerda ikki yo'l: (1) hozirgi UI ogohlantirish bilan saqlash, (2) soft-delete'da cascade restore qilish va admin tasdiqlashi.

13.5 Tax hisoblash olib tashlanishi

Holat:

Tax butunlay olib tashlangan, lekin UI'da statik info'lar (12%, 4%, 2%) saqlanib qolgan.

Tavsiya:

agar bu deductionlar real hisoblanishi kerak bo'lsa, ulardan birini (yoki barchasini) qaytarish uchun yangi modul ishlab chiqish kerak.

13.6 Initial Balance — birdan ko'p taroflar uchun

Holat:

Yangi tizimga o'tkazilayotganda eski tizimdagi *balansga zid bo'lgan* qarzdorlikni qanday hisoblash aniq emas. Hozir `amount >= 0` validatsiya bor.

Tavsiya:

qarzdor o'quvchilar uchun `ADJUSTMENT` bilan manfiy balans yozish mumkin (CEO/BD ruxsati bor).

13.7 Davomatni o'qituvchi qo'lda olishi va frauder

Holat:

Tizim o'qituvchining davomatni qo'lda olishi va kim olganini farqlamaydi (markedMethod). O'qituvchi o'z foydasi uchun yolg'on davomat qo'yishi mumkin.

Tavsiya:

Audit log + per-attendance markedById ko'rsatish (allaqachon EntityHistory'da bor). QR davomatni majburiy qilish (ya'ni o'qituvchi qo'lda PRESENT qo'ya olmasligi) — alohida sozlama bo'lishi mumkin.

13.8 perLessonCost snapshot izchilligi

Yaxshi xususiyat:

LESSON_DEDUCTION metadata'sida `perLessonCost` snapshot saqlanadi. Kurs narxi keyin o'zgarsa ham, asl tsikl narxi saqlanadi va refund hisoblashda ishlatiladi. Bu — to'g'ri yondashuv.

14. Xulosa

DaF ERP tizimining moliyaviy quyi tizimi **append-only ledger** printsipli asosida qurilgan, atomiklik **Serializable** izolatsiya darajasi bilan kafolatlanadi va har bir *aktiv* yozuv `reversedAt IS NULL` filtri orqali aniqlanadi. Asosiy konseptlar:

- **Prepaid Model:** birinchi davomatda butun tsikl uchun pul yechiladi (FULL_CYCLE), keyin har dars prepaid hisobchidan kamayadi
- **B.1 Coverage qoidasi:** ustozga oylik faqat o'quvchi to'lov qilgan dars uchun yoziladi
- **SCD2 Versioning:** oylik konfiguratsiyalari va davr sozlamalari tarixi saqlanadi
- **Atomik cascade:** guruhdan chiqish, transfer, dars bekor qilish — barchasi bir tranzaksiyada
- **Rolga asoslangan ruxsatlar:** CEO eksklyuziv operatsiyalari (oylik tasdiqlash, hisoblash davri, soliq, lesson deduction reverse) bekor qilib bo'lmaydi

Tizim hozirda **892/892 backend testdan o'tadi** va frontend ham build'dan o'tadi (CLAUDE.md ga ko'ra). Bu — production deploy uchun yetarlicha tayyorlik darajasi.

Ushbu hujjat foydalanuvchi tomonidan so'ralgan barcha holatlarni qamrab oldi:

- ☒ Pul tushishi va yechilishi (3 xil holat: standart, intensiv, partial)
- ☒ Ustozga oylik (3 turi: PERCENTAGE, FIXED_PER_STUDENT, FIXED_MONTHLY)
- ☒ O'quvchi guruhdan chiqsa nima bo'ladi (refund prepaid)
- ☒ Yangi o'quvchi qo'shilsa (enroll va birinchi davomat)
- ☒ A guruhdan B guruhga ko'chsa (transfer)
- ☒ Bir guruhda ustoz o'zgarsa (GroupTeacherHistory)
- ☒ Muzlatish (FROZEN) va to'lov muddatining o'zgarishi
- ☒ Eski tizimdan migratsiya (INITIAL_BALANCE)
- ☒ Dam olish kunlari va dars bekor qilinishi
- ☒ Ustozning real vs kutilayotgan oyligini ko'rishi
- ☒ Effektiv sanadan boshlab oylik o'zgarishi
- ☒ Refund, payment reverse, lesson deduction reverse
- ☒ Edge case'lar (B.1 qoidasi, race condition, concurrent davomat)